

KANned neural networks

Florent de Dinechin

September 24, 2025

Abstract

The identity $a \times b = ((a+b)^2 - (a-b)^2)/4$ rewrites a multiplication as a fixed-size combination of additions and unary functions. Along with a few others, this identity provides a constructive technique to transform a neural network from the mainstream families (multi-layer perceptron, deep convolutional neural network, and transformers) into a mathematically equivalent multi-layer Kolmogorov-Arnold Network (KAN) with the same structure. The resulting KAN activation functions are piecewise linear functions, simple degree-2 polynomials, reciprocal and exponential. They may be non-smooth, which is an invitation to relax some constraints which are taken for granted in the recent KAN literature. This technique is essentially of theoretical interest, but practical applications are also discussed.

1 Introduction

In all the sequel, KAN refers to multi-layer Kolmogorov-Arnold Networks as presented by Liu et al.[2], with one big caveat: we may use activation functions which are out of the scope of this paper. This relaxation will be discussed in Section 5.3.

2 All MLPs are KAN

A multi-layer perceptron (MLP) can trivially be interpreted as a KAN. Figure 1 is a classical representation of a perceptron layer, where the multiplications by weights have been replaced with unary functions.

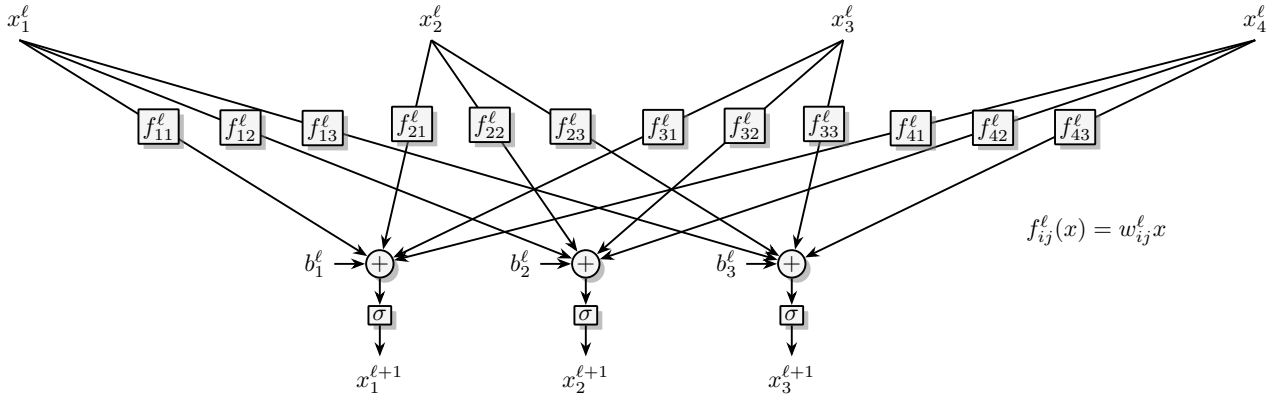


Figure 1: An MLP layer is almost a KAN layer

The conversion of an MLP to a multi layer KAN simply consists in fusing the activation function from the initial MLP (the σ in Fig. 1) and the input weight functions of the next layer. This is a simple function composition: we obtain a KAN network whose functions are all of the form $g_{ij}^\ell(x) = f(\sigma(x + b))$. In the process we also fuse the bias input to the adder (another option would be to have it output by a constant KAN activation function)

The resulting network has the structure of a dense multi-layer KAN. An interesting observation is that the resulting function is continuous but not necessary smooth, which means that the KANs thus obtained may not belong to the class of KANs being trained with current techniques. This will be discussed in Sect. 5.3

This work was partially supported by the PEPR IA HOLIGRAIL project of the Agence Nationale de la Recherche, ANR-23-PEIA-0010

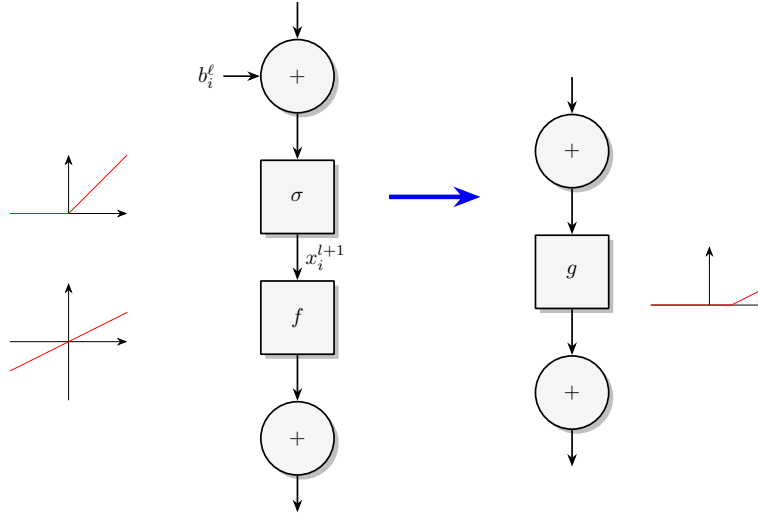


Figure 2: MLP function fusion

It is important to note that this technique transforms each MLP layer into a KAN layer implementing exactly the same function (no approximation here). The depth (number of layers) of the KAN network is that of the original MLP. Each layer has the same number of adders. The number of activation functions in the KAN is the number of weights in the MLP.

3 All CNNs are KANs

Let us list the other layers that make up most convolutional neural networks.

- Obviously the previous conversion can be applied to convolution layers.
- The same is true of average pooling layers (weighted average or not).
- Batch normalization layers are useful in training. However, after training, they become simple pointwise affine functions which can either be merged in the surrounding layers, or converted to KAN layers by the previous transformation.
- Skip connections are identity functions.
- Altogether, the only layer type of practical significance that resists the previous transformation to KAN is the max-pooling layer.

It can still be converted to additions and unary functions thanks to the following identity:

$$\max(a, b) = \frac{a + b + \text{ReLU}(a - b) + \text{ReLU}(b - a)}{2}. \quad (1)$$

Since we have additions before and after the ReLU function, this requires two layers of KAN. For a max-pool on a window of 2×2 window we have to evaluate $\max(\max(a, b), \max(c, d))$. This means two application of (1) entailing 4 layers of KAN. In general, max-pooling with a window containing N values can be performed with a binary tree of depth $\lceil \log_2 N \rceil$ hence $2 \lceil \log_2 N \rceil$ layers of KAN.

Of course this transformation is of purely theoretical interest, in practice we would still use the max function.

Finally, any CNN can be transformed into an algebraically equivalent KAN which has exactly the same structure: same depth (plus a few units if it involves max-pooling), same width of each layer, and same connectivity.

4 All transformers are KANs

Transformers are essentially composed of linear functions (matrix-matrix products) and SoftMax.

4.1 Linear layers

The matrix-matrix products reduce to parallel dot-products:

$$\forall i, j \quad c_{ij} = \sum_k a_{ik} b_{kj} \quad (2)$$

Figure 3 shows how one product can be transformed into unary functions and additions using the identity $4ab = (a+b)^2 - (a-b)^2$.

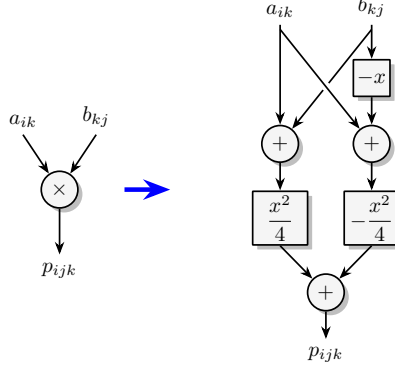


Figure 3: Two-input multiplication conversion

With this, a sum of N products can be converted to two KAN layers of respective sizes $2N$ and N adders and as many activation functions.

4.2 Softmax

The softmax function inputs a vector (x_i) and outputs a vector (y_i) of the same size N defined as

$$\forall i \quad y_i = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}} \quad (3)$$

There, the division a/b can be transformed into a product and a (unary) reciprocal function: $a \times \frac{1}{b}$.

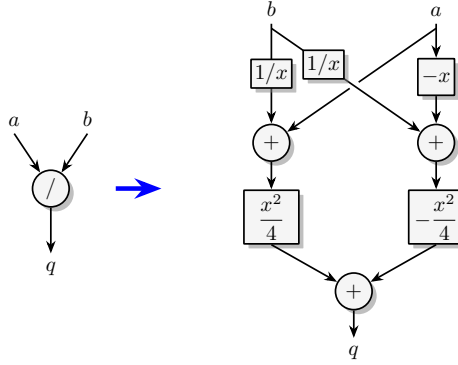


Figure 4: Conversion of the division a/b

The exponential e^x is already a unary function. Altogether, a softmax can be rewritten as three KAN layers.

Some of these layers have as activation functions the reciprocal (used only on the positive domain, so avoiding its discontinuity), and the exponential. Again, these functions don't belong to the pool of functions that current KAN research trains for. However, they are well-defined, well-behaved functions, so we feel we may claim that the transformed network is a KAN.

A practical issue is that the exponential function goes quickly to infinity. For this reason, softmax is in practice better implemented as "stable softmax", an algebraic rewriting of softmax that ensures that the exponential is only invoked on negative arguments. However it involves computing the maximum value of the input vector, which can be expressed as KAN layers (similar to max-pooling layers) but adds to the depth.

5 Discussion

5.1 Multi-layer KAN are stronger than multi-layer perceptron

Any CNN can be converted to a KAN of the same size which (assuming exact arithmetic) has the exact same behaviour. As we have only used a very small subset of the possible KAN functions, the opposite is not true. This is another trivial proof that KANs are stronger than MLPs and CNNs... if we can train them.

5.2 Practical applications?

Indeed, this work could help bootstrap the training of KAN networks by starting with a pretrained CNN or transformer, and then relaxing the activation functions. We have on huggingface thousands of network that can be converted to KANs that already work. This could be a productive way to transfer to KANs some knowledge accumulated on CNNs and transformers (network organisation, hyperparameters).

5.3 KAN beyond splines could be useful

In most of the recent KAN literature, all the activation functions are trained in such a way that they are smooth (i.e., continuous with continuous derivative). This is the main motivation for using splines, Chebyshev polynomials, or squared ReLUs [3]. This choice seems mostly justified by the necessity to learn smooth functions from physics, and also to avoid untrainable functions that would make KAN “irrelevant” [1].

The composition of two continuous functions is continuous. Activation functions in classical networks are always continuous (a necessity for training). However, if σ was not smooth, $f_{\oplus}^{w_i} \circ \sigma$ is not smooth either, as illustrated in Fig. 2 in the case when $\sigma = \text{ReLU}$: there, one observes a discontinuity in the derivative inherited from the discontinuity in ReLU. In this sense, the KANs that we obtain do not belong to the space explored by current KAN training approaches.

This observation can be considered in two different ways.

- If one of our readers has a compelling argument for insisting that KAN activation functions should be smooth, one may always replace ReLU with one of its smooth variants, such as $\text{GELU}(x) = \frac{x}{2}(1 + \text{erf}(\frac{x}{\sqrt{2}}))$ or $\text{SiLU}(x) = \frac{x}{1+e^{-x}}$ – both of which can be scaled at will to make them arbitrarily close to ReLU.
- Conversely, we have here a constructive technique that rewrites a working CNN into a mathematically identical KAN with non-smooth activation functions. In other words, the proposed technique exhibits a huge set of KANs with non-smooth functions that have already been demonstrated useful in practice. This seems a compelling argument for relaxing the constraint that KAN activations should be smooth. The successful use of ReLU in training CNNs tends to suggest that this constraint is not necessary for training.
- Finally, the next step will be to quantize KANs to deploy them on low-precision hardware. Weights quantized on 4 bits or less are extremely discrete, and even the notion of continuity makes little sense at this scale. In other words, an important challenge ahead is quantization-aware training with KANs (KAN-QAT), and the definition of a good function basis should also take quantization into account.

6 Conclusion

This work could be described as KAN engineering as opposite to KAN training.

An interesting remark is that none of this work makes any use of the Kolmogorov approximation theorem.

References

- [1] Federico Girosi and Tomaso Poggio. “Representation properties of networks: Kolmogorov’s theorem is irrelevant”. In: *Neural Computation* 1.4 (1989), pp. 465–469.
- [2] Ziming Liu et al. *KAN: Kolmogorov-Arnold Networks*. 2025. arXiv: 2404.19756 [cs.LG]. URL: <https://arxiv.org/abs/2404.19756>.
- [3] Qi Qiu, Tao Zhu, Helin Gong, Liming Chen, and Huansheng Ning. *ReLU-KAN: New Kolmogorov-Arnold Networks that Only Need Matrix Addition, Dot Multiplication, and ReLU*. 2024. arXiv: 2406.02075 [cs.LG]. URL: <https://arxiv.org/abs/2406.02075>.